

RUBY

Классы · Модули · Коллекции · Организация кода

Команда 5

Тимошенко Дарья
Андреев Ярослав
Гиниятуллина Аида



Ярослав



Дарья



Аида

02 презентация

Что разберём

Во второй презентации показываем, как из Ruby-конструкций собирается обычная программа

Объектная модель

- классы и объекты
- атрибуты
- наследование
- методы класса

Модули

- пространства имён
- mixins
- подключение общего поведения

Работа с данными

- Array, Hash, Set
- Enumerable
- duck typing
- pattern matching

Организация проекта

- require и require_relative
- gems и Bundler
- структура файлов
- итоговый пример

Классы и объекты

Класс описывает состояние и поведение объектов

- `class` — объявление класса, `new` — создание нового объекта.
- `initialize` вызывается автоматически при создании объекта.
- Переменные экземпляра начинаются с символа `@` и хранят состояние объекта.
- Метод `passed?` возвращает логическое значение и использует правило именования Ruby для предикатов.

```
class Student
  def initialize(name, score)
    @name = name
    @score = score
  end

  def passed?
    @score >= 60
  end
end

student = Student.new("Anna", 92)

puts student.passed?
```

03_classes.rb

Вывод программы

true

Атрибуты объекта

Ruby позволяет быстро создать методы чтения и изменения атрибутов

- `attr_reader` создаёт `getter` и позволяет читать значение атрибута.
- `attr_writer` создаёт `setter`; `attr_accessor` объединяет чтение и изменение.
- В примере имя можно только прочесть, а возраст — и прочесть, и изменить.
- Это делает код короче и избавляет от ручного написания однотипных методов.

```
class User
  attr_reader :name
  attr_accessor :age

  def initialize(name, age)
    @name = name
    @age = age
  end
end

user = User.new("Alice", 20)
user.age += 1

puts user.name
puts user.age
```

04_attributes.rb

Вывод программы

Alice
21

Наследование

Дочерний класс расширяет поведение родительского

- Запись `Student < Person` означает, что `Student` наследуется от `Person`.
- Метод `super` позволяет вызвать реализацию из родительского класса.
- В `initialize` мы передаём имя родительскому классу, а затем добавляем собственное поле `group`.
- Метод `info` сначала берёт строку из родительского класса, а потом дополняет её данными о группе.

```
class Person
  def initialize(name)
    @name = name
  end

  def info
    "Person: #{@name}"
  end
end

class Student < Person
  def initialize(name, group)
    super(name)
    @group = group
  end

  def info
    "#{super}, group: #{@group}"
  end
end

puts Student.new("Max", "6767").info
```

05_inheritance.rb

Вывод программы

Person: Max, group: 6767

Методы класса и self

Не все методы принадлежат объекту — некоторые относятся к самому классу

- Метод класса объявляется через `self.method_name`.
- Такой метод удобно использовать для утилитарных операций, когда отдельный объект создавать не нужно.
- В примере метод преобразует температуру и вызывается как `Temperature.celsius_to_fahrenheit(20)`.
- Значение `self` обозначает текущий объект и зависит от контекста выполнения.

```
class Temperature
  def self.celsius_to_fahrenheit(celsius)
    celsius * 9 / 5.0 + 32
  end
end

puts Temperature.celsius_to_fahrenheit(20)
```

06_class_methods.rb

Вывод программы

68.0

Модули и пространства имён

Модули помогают группировать код и избегать конфликтов имён

- Module может содержать классы, методы, константы и другие модули.
- Оператор `::` используется для обращения к вложенному классу или константе.
- В примере `Student` находится внутри модуля `University`, поэтому полное имя класса — `University::Student`.
- Такой подход удобен для крупных проектов и библиотек.

```
module University
  class Student
    def initialize(name)
      @name = name
    end

    def label
      "Student: #{@name}"
    end
  end
end

student = University::Student.new("Anna")
puts student.label
```

07_namespace.rb

Вывод программы

Student: Anna

Mixins

Общее поведение можно подключать к классам через модули

- Модуль Printable содержит метод `print_info`.
- После `include Printable` этот метод становится доступен объектам класса `Report`.
- Mixin позволяет переиспользовать поведение без необходимости строить длинную иерархию наследования.
- Это одна из характерных возможностей Ruby.

```
module Printable
  def print_info
    puts info
  end
end

class Report
  include Printable

  def info
    "Report: Ruby"
  end
end

Report.new.print_info
```

08_mixins.rb

Вывод программы

Report: Ruby

Коллекции глубже

Array, Hash и Set покрывают разные сценарии хранения данных

- Array хранит упорядоченную последовательность элементов.
- Hash хранит пары ключ → значение и хорошо подходит для словарей и структурированных данных.
- Set содержит только уникальные элементы, поэтому повторяющиеся значения автоматически убираются.
- В примере массив расширяется, возраст в Hash меняется, а Set оставляет только уникальные числа.

```
require "set"
```

```
languages = ["Ruby", "Python"]  
languages << "Go"
```

```
user = { name: "Alice", age: 20 }  
user[:age] += 1
```

```
numbers = Set.new([1, 1, 2, 3])
```

```
p languages
```

```
p user
```

```
p numbers
```

09_collections.rb

Вывод программы

```
["Ruby", "Python", "Go"]  
{:name=>"Alice", :age=>21}  
#<Set: {1, 2, 3}>
```

Enumerable

Коллекции можно обрабатывать цепочками небольших операций

- `select` выбирает элементы по условию, `map` преобразует каждый элемент, `sum` суммирует результат.
- В примере мы сначала получаем только чётные числа.
- Потом превращаем их в квадраты и в конце находим сумму этих квадратов.
- Такой стиль даёт читаемое и последовательное описание обработки данных.

```
numbers = [1, 2, 3, 4, 5, 6]

evens = numbers.select(&:even?)
squares = evens.map { |n| n**2 }
total = squares.sum

p evens
p squares
puts total
```

10 enumerable.rb

Вывод программы

```
[2, 4, 6]
[4, 16, 36]
56
```

Duck typing

В Ruby важнее поведение объекта, чем его конкретный класс

- Метод `print_name` ничего не знает о классах `User` и `Product`.
- Ему достаточно, что переданный объект умеет отвечать на сообщение `name`.
- Поэтому один и тот же код можно использовать с объектами разных классов.
- Такой подход делает Ruby гибким и поддерживает написание более обобщённого кода.

```
User = Struct.new(:name)
Product = Struct.new(:name)

def print_name(object)
  puts object.name
end

print_name(User.new("Alice"))
print_name(Product.new("Ruby Book"))
```

11_duck_typing.rb

Вывод программы

Alice
Ruby Book

Pattern matching

Конструкция case / in проверяет структуру значения и сразу извлекает данные

- Ruby сопоставляет значение с шаблоном и выбирает подходящую ветку.
- В примере проверяется Hash с полями name и score.
- Если score попадает в диапазон 90..100, программа сразу выводит оценку excellent.
- Переменные name и score извлекаются прямо внутри шаблона.

```
student = { name: "Daria", score: 92 }

case student
in { name:, score: 90..100 }
| puts "#{name}: excellent"
in { name:, score: 60..89 }
| puts "#{name}: passed"
else
| puts "Not passed"
end
```

12 pattern_matching.rb

Вывод программы

Daria: excellent

Открытые классы

Уже существующий класс можно открыть повторно и добавить ему новые методы

- Ruby позволяет изменять определение класса после его первоначального объявления.
- В примере классу String добавляется метод shout.
- После этого метод можно вызывать у любых строк.
- Такой приём полезен, но им нужно пользоваться аккуратно: глобальные изменения могут повлиять на поведение сторонних библиотек.

```
class String
  def shout
    upcase + "!"
  end
end

puts "ruby".shout
```

13 open_classes.rb

Вывод программы

RUBY!

Разделение программы на файлы

Собственный код обычно подключают через `require_relative`

- Когда проект растёт, неудобно хранить всё в одном файле.
- Модуль `Formatter` вынесен в отдельный файл `14_formatter.rb`.
- Файл `14_main.rb` подключает его через `require_relative` и затем вызывает метод `Formatter.title`.
- Такой подход позволяет разделить обязанности между файлами и сделать структуру проекта понятнее.

```
module Formatter
  def self.title(text)
    "== #{text.upcase} =="
  end
end
```

14_formatter.rb

```
require_relative "14_formatter"

puts Formatter.title("ruby")
```

14_main.rb

Вывод программы

== RUBY ==

Gems и Bundler

Внешние библиотеки Ruby обычно подключаются через Gemfile и Bundler

- Gem — пакет с Ruby-кодом и метаданными. Зависимости проекта перечисляются в Gemfile.
- В примере подключается gem `colorize`.
- Команда `bundle install` устанавливает зависимости и формирует `Gemfile.lock` с выбранными версиями.
- Это помогает всей команде работать с одинаковым набором библиотек.

```
source "https://rubygems.org"  
  
gem "colorize"
```

15_Gemfile

Основная команда

`bundle install`

Альтернативно

`bundle add colorize`

Структура проекта

Даже в небольшом Ruby-проекте лучше заранее разделить код по ролям

Пример структуры

```
02/
├── Gemfile
├── Gemfile.lock
├── lib/
│   ├── university.rb
│   └── university/
│       ├── student.rb
│       └── report.rb
└── main.rb
```

Зачем так делать

- lib/ — основная логика проекта: модули, классы и связанные с ними файлы.
- main.rb — точка запуска программы.
- Gemfile — зависимости, а Gemfile.lock — зафиксированные версии.
- Когда каждый файл отвечает за одну понятную часть программы, код легче читать и поддерживать.

Итоговый пример: модель

Объединяем module, mixin, класс и атрибуты в одну структуру

- Модуль Passable содержит общее правило: студент считается прошедшим, если его score не меньше 60.
- Класс Student подключает это поведение через include Passable.
- Через attr_reader создаются методы чтения для name и score.
- Таким образом, данные и общее поведение разделены, но при этом хорошо сочетаются.

```
module Passable
  def passed?
    score >= 60
  end
end

class Student
  include Passable

  attr_reader :name, :score

  def initialize(name, score)
    @name = name
    @score = score
  end
end
```

17_model.rb

Итоговый пример: отчёт

Финальная программа строит список имён студентов, прошедших порог

- Класс Report получает коллекцию объектов Student.
- Метод passed_names использует select и map — сначала отбирает студентов по правилу passed?, затем извлекает их имена.
- В примере участвуют три студента: Anna, Max и Eva.
- После фильтрации остаются только Anna и Eva, потому что их score не меньше 60.

```
require_relative "17_model"

class Report
  def initialize(items)
    @items = items
  end

  def passed_names
    @items.select(&:passed?).map(&:name)
  end
end

students = [
  Student.new("Anna", 92),
  Student.new("Max", 54),
  Student.new("Eva", 87)
]

report = Report.new(students)

p report.passed_names
```

18 final_demo.rb

Вывод программы

["Anna", "Eva"]

Итоги второй презентации

Теперь видно, как из Ruby-конструкций складывается более крупная программа

Что разобрали

- классы и объекты
- атрибуты и наследование
- методы класса и self
- модули и mixins
- Array, Hash, Set
- Enumerable
- duck typing
- pattern matching
- открытые классы
- require и require_relative
- gems и Bundler
- структуру проекта
- итоговую модель Student
- класс Report и обработку коллекции

Спасибо за внимание!